SOFTWARE ARCHITECTURE — CHAPTER 8

Compatibility and Coupling

How dependencies bind systems—and how to evolve contracts safely

Based on Pautasso — Visual Lecture Notes

Chapter PDF: `08_Compatibility_and_Coupling.pdf`

RECAP — LECTURE 7

Composability & Connectors

Chapter 7: how we assemble components and choose how they communicate

Examples: `example1_composition_and_connectors.py`, `example2_orchestration_vs_choreography.py`

→ Today: how strong those connections are (**coupling**) and whether we can change them safely (**compatibility**).

Recap: what is composition?

Composition = building a system by assembling smaller **components** rather than one monolithic block.

- Each component has a clear responsibility
- Components are connected by **connectors**
- Architecture ≈ **components + connectors**

Every connector implies shared assumptions—those assumptions are what Chapter 8 calls a **dependency surface**.

Recap: connectors

A **connector** is the mechanism that allows communication between components.

- Defines *protocol*, *direction*, and often *sync vs async*
- Same component can use different connectors to different neighbors
- Choosing the wrong connector affects **coupling**, latency, and failure modes

Recap: connector types (overview)

Type	Typical use	Often
Direct call	Same process, in-memory	Sync
RPC / REST	Services over network	Sync (request/response)
Message queue	Decouple producer/consumer	Async
Event bus	Many subscribers, pub/sub	Async
Shared DB	Implicit coupling via data	Sync read/write

Later: shared schemas and deployment often create the **strongest** coupling.

Recap: message passing

Synchronous

Caller sends a message and **waits** for a reply.

Simple mental model; tight temporal coupling; blocking under load.

Asynchronous

Caller sends and **continues**; consumer processes later.

Better decoupling and scale; harder reasoning; **event schema** still couples publishers and subscribers.

Recap: orchestration vs choreography

	Orchestration	Choreography
Control	Central	Distributed
Visibility	High (one place)	Lower (many handlers)
Scaling	Orchestrator matters	Workers scale independently
Good for	Clear workflows	Many reactive consumers

Both need explicit rules when payloads, ordering, or error semantics change.

Recap: event-driven & bridge

- **Event** — something happened (e.g. `OrderPlaced`); broker delivers to subscribers
- Watch for: ordering, duplicates, eventual consistency, **schema evolution**
- Lecture 7: **topology** of communication · Lecture 8: **strength** of dependencies & **safe evolution**
- Builds on Lecture **6**: interfaces, contracts, semantic versioning, deprecation

Learning objectives

- Explain **coupling** between components and clients—and why it drives cost of change
- Name common coupling dimensions (data, control, temporal, deployment) and spot them in diagrams
- Distinguish **syntactic** vs **semantic** compatibility on APIs and events
- Classify **breaking** vs **non-breaking** changes and map them to **semver**
- Compare **version coexistence** strategies and their operational trade-offs
- Apply **boundaries**, ports/adapters, and policies to keep evolution manageable

What is coupling?

Coupling is how much one part of a system **depends on** concrete details of another (API shape, behavior, timing, deployment, or data layout).

- **Strong** coupling → a small change can **ripple** across modules, teams, or external integrators
- Dependencies are unavoidable; the goal is to make them **visible**, **directional**, and **affordable**
- Architectural views (component, deployment, runtime) should show *where* coupling concentrates

Coupling is the “resistance” you feel when one team cannot ship without coordinating many others.

Why coupling matters

Coupling is a first-class driver of **quality attributes** you care about in practice.

- **Change velocity** — high coupling slows refactors and safe rollouts
- **Testability** — hard to isolate units when everything reaches into shared structures
- **Reliability** — sync chains and shared failure domains amplify outages
- Shared databases, generated DTOs, and “reuse internal models in the API” are classic **coupling magnets**

Reviews should ask: “If we change X, who *must* change in the same release train?”

Kinds of coupling (overview)

Kind	Idea	Example signal
Data	Shared schemas, DB tables, DTOs	Column rename breaks many services
Control	Who invokes whom, call order	Private “call A before B” convention
Temporal	Must be up together, sync RPC	Cascade timeouts under load
Deployment	Release or scale as one unit	Cannot deploy API without worker
Platform / toolchain	Shared runtime, codegen, SDK lockstep	Clients pinned to old protobuf plugin

Align wording with your chapter PDF taxonomy; the point is to **label** edges in your diagrams.

Reducing coupling

- **Interfaces / ports** — depend on abstractions; swap adapters for DB, queue, or HTTP (Example 2)
- **Anti-corruption layer** — translate external models at the boundary; don't leak vendor types inward
- **Versioned public API** — stable façade over evolving internals
- **Feature flags** — decouple *code present* from *behavior exposed*
- **Tolerant reader** — ignore unknown fields; optional new data (Example 1)

Events reduce some temporal coupling but move coupling to **schema contracts** and consumer logic.

Cohesion (brief)

Cohesion = how closely the responsibilities inside *one* module belong together.

- High cohesion + **loose** coupling across modules is usually easier to evolve
- Low cohesion often shows up as “god services” that drag unrelated callers into the same release
- When splitting services, aim for clear boundaries—not just smaller deployables

Coupling is *between* parts; cohesion is *within* a part. Both affect change safety.

Compatibility

Parts are **compatible** when one side can evolve without forcing an immediate, coordinated change on every consumer.

- **Backward** — new server / producer, **old** clients still correct (primary goal for public APIs)
- **Forward** — old server, **new** clients (limited; needs feature detection, fallbacks, or negotiation)
- **Transitional** — dual endpoints, aliases, or adapters during migration windows

“Compatible” is defined relative to **documented** client assumptions—not only JSON Schema validity.

Syntactic vs semantic

Syntactic

Wire-level shape: field names, types, required vs optional, enums, headers.

Tools: JSON Schema, OpenAPI, protobuf definitions.

Semantic

Meaning and behavior: validation ranges, ordering guarantees, idempotency, error `code` semantics, side effects.

Same payload can imply different behavior after a “compatible” deploy.

Example: stricter validation can be a **silent** break even when the schema file barely changed.

Breaking vs non-breaking (typical)

Often non-breaking

Add **optional** field or query param with a safe default.

Add **new** resource or endpoint; widen accepted values if previously undocumented.

Often breaking

Rename/remove **required** fields; new **required** headers or body fields.

Tighten validation; change default behavior; remove endpoints.

Golden question: *Would a deployed old client still succeed without code or config changes?*

Semantic versioning (reminder)

- **MAJOR** — changes that are incompatible for *existing* consumers under your stated rules
- **MINOR** — backward-compatible **additive** functionality
- **PATCH** — backward-compatible **bug fixes**
- Pre-release tags (`-beta`) signal instability; document what “compatible” means for your API

Semver only helps if your **policy** matches real client parsers (strict vs tolerant).

Version coexistence

- **URL paths** — `/v1/tasks` vs `/v2/tasks`; easy for partners to reason about
- **Headers / content negotiation** — `Accept-Version`, vendor media types; needs disciplined gateways
- **Separate deployments** — different clusters or hostnames; strongest isolation, highest ops cost
- **Sunset** — published dates, metrics on remaining v1 traffic, escalation for stragglers

Dual stacks pay rent: routing, auth, observability, security patches on *both* code paths.

Dependencies & layering

Keep **policy and domain** from depending on low-level infrastructure details (ORM, message SDK, HTTP client quirks).

- **Dependency inversion** — inner layers define ports; outer layers provide adapters
- Improves test doubles, alternative implementations, and incremental rewrites
- Public API models need not equal internal entity graphs—translate at the edge

Walk through `example2_coupling_and_dependencies.py`: tight dict vs `TaskRepository`.

Contracts in practice

- Treat OpenAPI / AsyncAPI / schema repos as **products** with owners and changelogs
- **Consumer-driven contracts** — tests prove providers still satisfy real client expectations
- **Postel's principle** — conservative in what you send, liberal in what you accept (within security bounds)
- Document error envelopes: stable `code`, evolving `message` strings

Pair with Example 1: lenient parse + additive response fields.

API & schema evolution patterns

- Additive JSON fields with defaults; avoid reusing field names for new meanings
- Deprecation headers (`Deprecation` , `Sunset`) + docs + metrics
- Parallel fields during rename (`done` + `completed`) before removing the old one
- Feature flags to expose behavior only after clients are ready
- For events: schema registry, compatibility modes (backward/forward), replay testing

Patterns reduce pain but do not remove the need for a written **policy** (see assignment Part 3).

Course examples (Python)

- `example1_compatibility_and_api_evolution.py` — classifies changes; optional `priority`; legacy v1 client reading v2 JSON; lenient server parsing older bodies
- `example2_coupling_and_dependencies.py` — **tight** coupling to a dict row shape vs **loose** coupling through `TaskRepository`

Run both after class; use `08_Compatibility_and_Coupling.pdf` for definitions and extended examples from the book.

ASSIGNMENT

Task Board API

Coupling analysis, compatibility classification, coexistence, policy

Full spec: `ASSIGNMENT.md` — this section orients you; it does **not** solve the assignment.

Assignment: scenario

A **Task Board REST API** serves a web SPA, mobile apps, and long-lived **partner integrations**.

- **Baseline (given):** clients → optional gateway → Task API → task store + notification service
- Product wants new fields, stricter validation, bulk operations, and better client attribution
- You produce **architecture documentation**: where coupling sits, how risky each change is, how versions coexist, and team rules for future changes

Design and documentation only — no production code required; diagrams + markdown in your PR folder.

What you produce (big picture)

Area	You decide & justify
Coupling	Inventory + dependency diagram with labels
Compatibility	Per proposed change: breaking? semver? semantic risk
Coexistence	How v1 and v2 run together during migration
Policy	Rules, deprecation, errors, partners
Diagram	Migration / dual-version sequence (≥5 participants)

Part 1 — Coupling inventory

Task 1.1 — written analysis

- List **≥5** elements (e.g. Web SPA, Mobile, Partner, Gateway, Task API, Store, Notifier)
- For each dependency: **direction**, coupling kind (data / control / temporal / deployment / …), what breaks if the provider changes
- Identify **two** tight couplings that are a **reasonable trade-off** and **two** you would loosen (how: façade, events, ACL, versioning, …)

Deliverable: `part1_coupling_analysis.md` — **30 pts**

Part 1 — Diagram

Task 1.2

- **draw.io** dependency or component diagram: boxes, arrows, labels (coupling type or “sync REST”, “shared schema”, …)
- Include a **legend** so a reviewer can read notation in under one minute
- Export **PNG** for the PR; keep **.drawio** source for edits

Deliverables: `part1_coupling_diagram.drawio` + `.png` — **20 pts**

Assignment: proposed changes (A–E)

	Change
A	Optional `priority` on `GET /tasks` response
B	Rename `done` → `completed`
C	Required header `X-Client-Id`
D	Shorten max `title` length 500 → 100
E	New `POST /tasks/bulk`

Assume **strict** mobile SDKs and **slow-moving** partners unless you state otherwise in your README.

Part 2 — Compatibility & semver

Task 2.1

- For **each A–E**: **breaking** vs **non-breaking** for existing deployed clients (justify with concrete failure modes)
- Recommend **MAJOR / MINOR / PATCH** bump for the *public* API package under semver rules **you** state
- One sentence of **semantic** risk per change (e.g. validation, idempotency, error handling)

Deliverable: `part2_compatibility_changes.md` — **25 pts**

Part 2 — Version coexistence

Task 2.2

- Pick a concrete **coexistence** strategy: URL version, header, gateway routing, or hybrid—say who implements it (edge, BFF, service)
- Describe how **new** clients adopt v2 while **legacy** traffic drains on v1 during a sunset window
- Name at least one **operational** cost: dual deploys, auth rules, observability, security patches, doc drift, support load

Deliverable: `part2_version_coexistence.md` — **15 pts**

Part 3 — Compatibility policy

Task 3.1

- ≤2 pages (markdown): rules for **additive** vs **breaking** changes; how **deprecation** is announced and how long it lasts
- How you keep **error** payloads stable for machines (codes) vs humans (messages)
- If **partners** get longer notice or different rules than first-party apps, say so explicitly

Deliverable: `part3_compatibility_policy.md` — **20 pts**

Part 3 — Migration sequence

Task 3.2

- **Sequence diagram** (draw.io): e.g. client hits gateway → routed to v1 or v2 → store; or partner migration with notifications
- **≥5 participants** and every message **labeled** (no anonymous arrows)
- Match the story you told in Part 2 (same actors, consistent names)

Deliverables: `part3_migration_sequence.drawio` + `.png` — **10 pts**

Submission & grading

- **Pull Request** to `lecture-8/submissions/YOUR_NAME/` — follow `../lecture-3/SUBMISSION_GUIDE.md`
- Include a short `README.md` in your folder: your name, what you focused on, any assumptions
- **Total: 120 points** — full rubric and quality bar in `ASSIGNMENT.md`
- Every diagram: submit **both** `.drawio` and `.png`

Before you start

Hints — not answers

1. Run `example1_compatibility_and_api_evolution.py` and `example2_coupling_and_dependencies.py`
2. Draw **clients → API → data**; highlight **shared DTOs** and **sync call chains** as coupling hotspots
3. For A–E, role-play: “Ship today—would production mobile binary or partner script still work?”
4. Cross-check your semver story with the policy you will write in Part 3 (no contradictions)

Takeaways

- **Coupling** is about dependency **strength** and **direction**—not only “do they talk?”
- **Syntactic** wire compatibility can mask **semantic** breaks (validation, defaults, errors)
- **Semantic versioning** works when teams share a precise definition of “compatible client”
- **Coexistence** strategies trade isolation for operational and documentation overhead
- **Boundaries** (ports, ACLs, versioned façades) turn big-bang releases into staged, measurable migrations

Questions?

Chapter 8 — Compatibility and Coupling

Deep dive: `08_Compatibility_and_Coupling.pdf`

Follow your course schedule for the next chapter and assignment deadlines.